TEXT
RAG

文本表示：从词袋到 BERT + RAG 数据基座

上午 • 四级台阶 + 向量实操 + 搭自己的 RAG 知识库

Day 3 上午 • 180 min • 把文字变成向量，再用 RAG 构建数据基座

深圳大学 • 经济学院

主讲：杨楷

kindyang.github.io

00

开场

今天上午要做什么

上午四件事（升级版）

- ① 把文字变成向量：词袋 → TF-IDF → Word2Vec → BERT（四级台阶）
- ② 亲手取向量：本地/在线两条路都走一遍
- ③ 用向量搭 RAG：四步流水线 + 桌面级 AnythingLLM
- ④ 下午接棒：用 LLM 直接抽取 + 合成报告

为什么还要读文本



数字告诉你发生了什么，文本告诉你他们怎么说

但文本是未经审计的自述——可靠性天然低于数字

（数字 = 发生了什么 • 文本 = 他们怎么说）

01

词袋与 TF-IDF

词袋实例 • n-gram • TF-IDF 手算

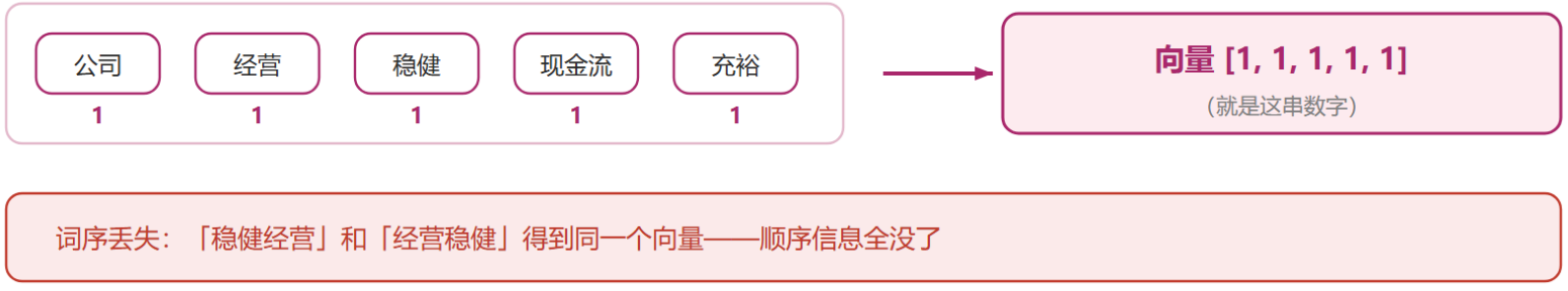
目标：让计算机能算「这两句话像不像」

- **计算机只认数字，文字必须先变成向量**
- 变成向量之后，「相似度」就是两个向量的夹角
- 接下来四级台阶，每一级都在解决上一级的一个毛病
- 词袋 → TF-IDF → Word2Vec → BERT

第一级：词袋模型

词袋模型：只数次数，不管顺序

例句：「公司经营稳健，现金流充裕」



词序丢失：「稳健经营」和「经营稳健」得到同一个向量——顺序信息全没了

词袋实例：两句「挺像」，意思却相反

- 句 A：「公司经营稳健，现金流充裕」→ $v_1=[1,1,1,1,1,0,0]$
- 句 B：「经营稳健，但现金流偏紧」→ $v_2=[0,1,1,1,0,1,1]$
- 重合词 3 个、长度相当 → 余弦相似度 ≈ 0.61，挺像
- 但「充裕」vs「偏紧」完全相反——词袋看不出来！

词袋的升级路线：n-gram 补一点顺序

- **二元组（bigram）：「稳健经营」和「经营稳健」算成不同特征**
- 三元组（trigram）再往上：顺序越来越精确
- 代价：组合爆炸——词表 1 万 → bigram 最多 1 亿
- 结论：n-gram 只是补丁，解决不了根本问题

TF-IDF 手算一例：「风险」vs「的」

词	篇内出现次数	含该词的文档数（共 100 篇）	TF-IDF 效果
「风险」	50 次	20 篇	IDF = log(100/20) 大 → 权重高
「的」	3000 次	100 篇	IDF = log(100/100) = 0 → 权重≈0
「公司」	800 次	95 篇	几乎每个文档都有 → 权重≈0

停用词表：先删掉「废话词」

- **通用停用词：的、了、在、和、是、及、与…**
- 财经专用停用词：公司、我们、本报告、管理层、年度…
- 删完再跑 TF-IDF，前十词才有信息量（回头看 P9 那页的条形图）
- 教训：前十词里冒出「公司」「我们」= 停用词没加够

先想通一件事：人，也能被「嵌入」成几个数

- 用 5 个维度（外向 / 严谨 / 温和 / 好胜 / 幽默）给每个人打坐标
- 张三 [0.8, 0.6, 0.7, 0.5, 0.4]，李四 [0.3, 0.8, 0.9, 0.2, 0.3]，王五 [0.7, 0.4, 0.3, 0.9, 0.8]
- 一算坐标距离：张三更像李四，还是更像王五？——立刻可分
- 词向量同理：把「稳健」压缩成 300 个数的坐标，词与词像不像 = 坐标距离



「King」的真实词向量：300 个浮点数

- **Word2Vec / GloVe 给每个词存一串 300 维浮点数（词嵌入向量）**
- King 的前 8 个数：0.5045, 0.6861, -0.5952, -0.0228, 0.6005, -0.1350, -0.0881, 0.4738
- 人看不懂这些数——但机器能算：谁离得近、方向朝哪
- 300 维是「恰到好处」的经验值：词表 20 万 × 300 维 ≈ 能存、能算、信息够

为什么稠密向量更好：one-hot vs 300 维

方案	维度	语义能力
one-hot	词表大小（10 万维）	任意两词都正交=无关；存不下、算不动
稠密向量	300 维（Word2Vec）	每个维度都有值；可度量远近、做运算

Word2Vec 怎么学：窗口与共现

- **窗口：前后各 2 个词（「公司 经营 稳健 现金流 充裕」逐个滑过）**
- 核心任务：用周围词预测中间词（CBOW）或反过来（Skip-gram）
- 预测对了 → 更新向量 → 共现多的词向量互相靠近
- 一句话直觉：你朋友的圈子定义你——distributional hypothesis

词袋的两个毛病

```
John likes to watch movies. Mary likes too.
John also likes to watch football games.
```

如果只需要表示出上面两句话中的单词，可以只对其中出现过的单词进行索引编码：

```
{"John": 1, "likes": 2, "to": 3, "watch": 4, "movies": 5, "also":6, "football": 7,
"games": 8, "Mary": 9, "too": 10}
```

其中的每个单词都可以用one-hot方法表示：

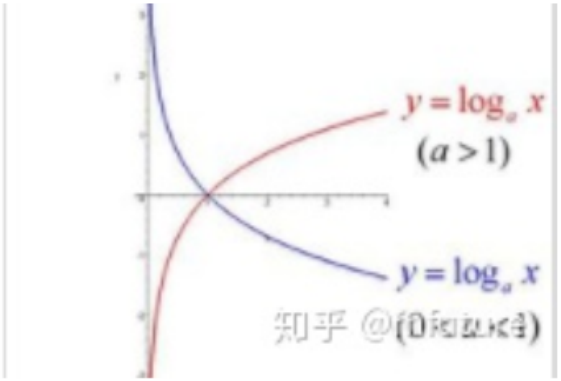
```
John: [1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
likes: [0, 1, 0, 0, 0, 0, 0, 0, 0, 0]
...
```

第二级：TF-IDF 的想法

- **光数词频不够，因为「的」「公司」到处都有**
- 思路：一个词在这篇里多不多，乘以它在所有文档里稀不稀有
- 在这篇里高频、在别处少见的词，才是这篇的特色词
- 这就压掉了那些通用的废话词

TF-IDF 的两个因子

一个词IDF值的计算是根据语料库得出的，如果一个词在语料库中越常见，那么分母就越大，IDF就越小越接近0。分母之所以要加1，是为了避免分母为0（即所有文档都不包含该词）。该词在语料库中越常见，IDF值越小。



看真实结果：年报 MD&A 的前十词

年报 MD&A 段 TF-IDF 前十词（某公司 2023 年报，jieba 分词）



反例：前十里「公司」「的」权重最低——停用词没加够的痕迹（教学对照）

02

Word2Vec 与 BERT

稠密向量 • 类比 • 上下文相关

第三级：Word2Vec 换个思路

- **前两级是「计数」，Word2Vec 改成「预测」**
- 分布式假说：一个词的意思，由它常跟哪些词一起出现决定
- 做法：拿中心词预测周围词（或反过来），训练完取权重当向量
- 结果：向量变成稠密的几百维，不再是稀疏的几万维

Word2Vec 怎么训练：一场「猜词」游戏

- **窗口滑过句子：中心词 + 左右各 2 个词组成训练样本**
- Skip-gram：用中心词「稳健」猜周边的「公司 / 经营 / 现金流 / 充裕」
- CBOW：反过来，用周边词猜中心词——两种架构，一种道理
- 猜错了就按差距微调词向量，反复迭代



负例采样：词表太大，全猜太贵

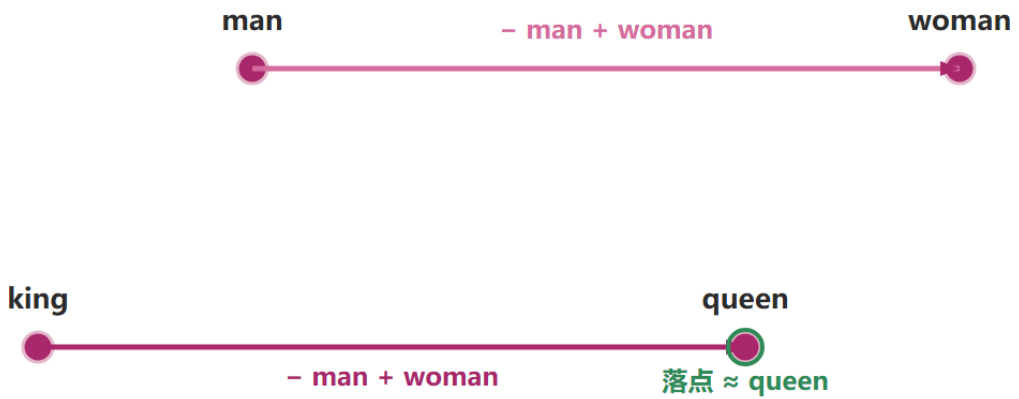
- **猜词要遍历整个词表（~20 万词）算概率再挑最大 ➜ 每猜一次都巨慢**
- 技巧：只跟「1 个正确答案 + 随机抽的 5-20 个错误答案」比
- 多分类（20 万选 1）变二分类（真邻居 / 假邻居）
- 随机抽的假邻居叫「负例」——这招叫负例采样（Negative Sampling）



一个著名的例子

词向量类比：king − man + woman ≈ queen

示意，实际是 300 维



向量运算能表达「类比关系」：国王减去男人加上女人，落点逼近女王

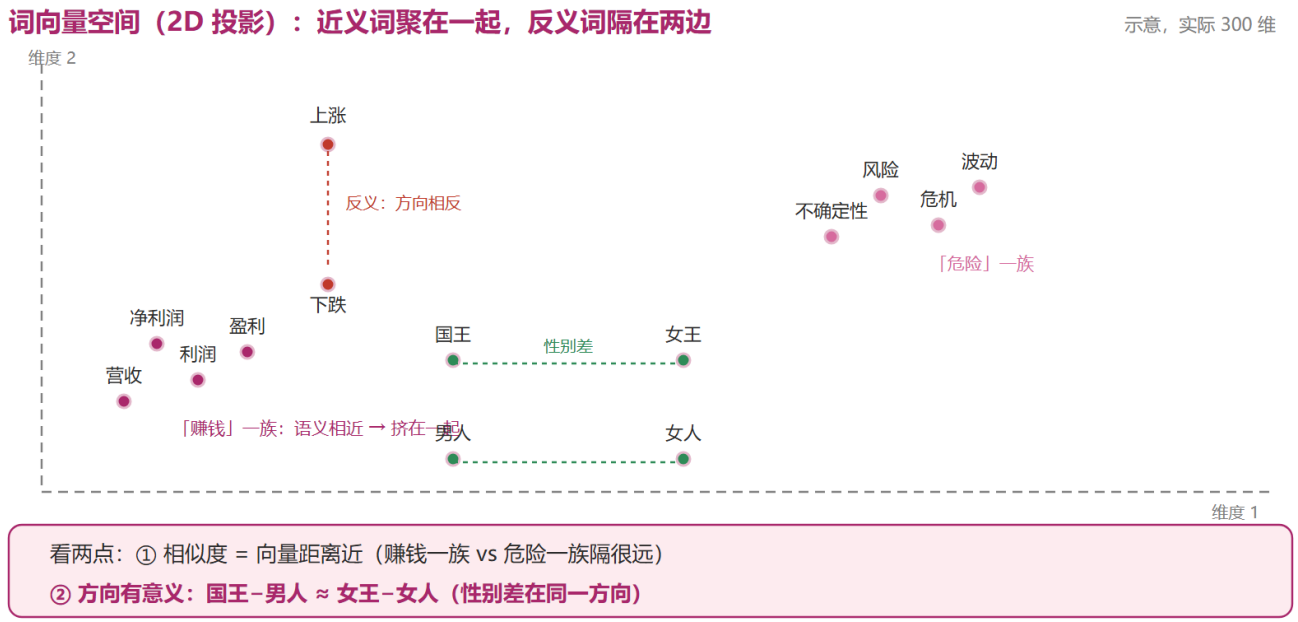
—— 语义关系被编码进了向量空间的方向

更多类比例子：位置关系也能算

类比	关系	说明
国王 − 男人 ≈ 女王 − 女人	性别	方向 = 性别轴
北京 − 中国 ≈ 巴黎 − 法国	首都	方向 = 首都轴
苹果 − 水果 ≈ 青菜 − 蔬菜	类别	方向 = 属类轴
夏天 − 热 ≈ 冬天 − 冷	属性	方向 = 温度轴

词向量二维图：近义词的距离一眼看清

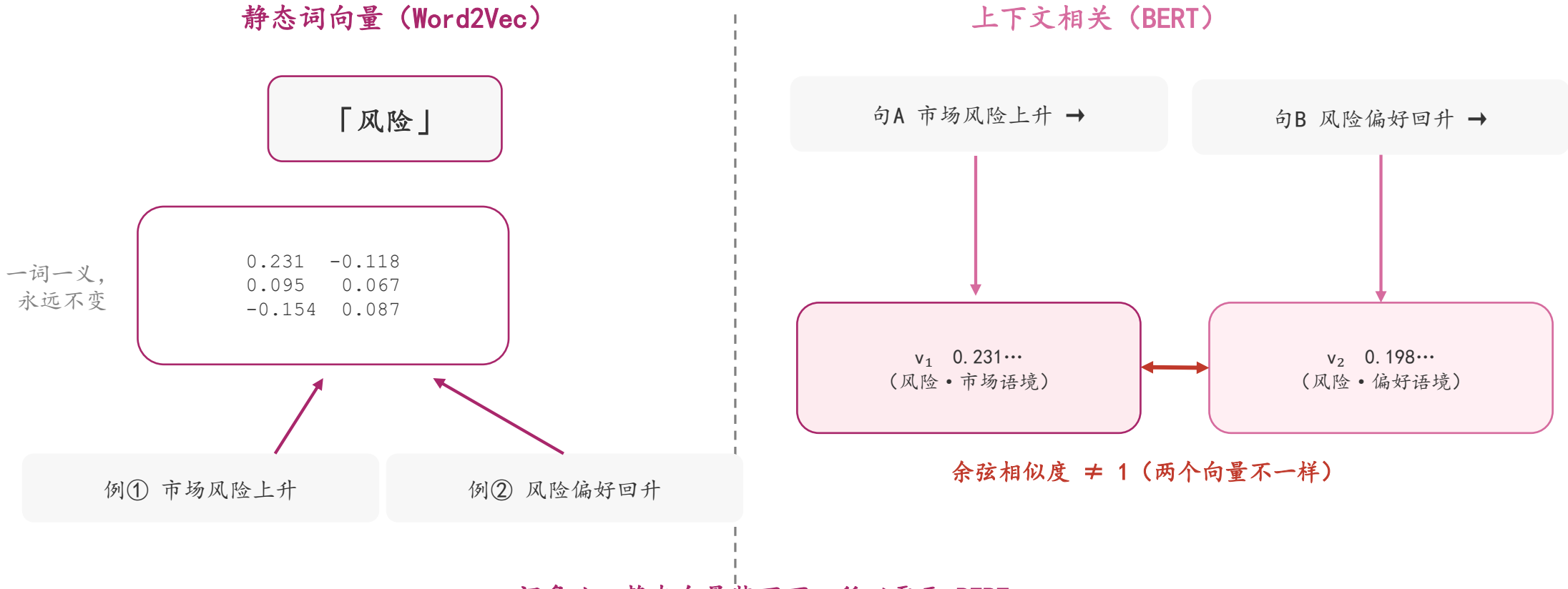
- **把 300 维压缩到 2 维投影（PCA/t-SNE），画在一张图上**
- 「赚钱」一族（净利润/利润/盈利）挤在一起；「危险」一族聚在另一边
- 反义词（上涨 vs 下跌）方向相反、距离相等
- 国王/女王/男人/女人构成平行四边形——关系方向一致



一词多义的失败案例：「银行」的两种意思

- 「把钱存进银行」 vs 「河岸（bank）边散步」——英文 bank 两义
- 中文同理：「银行放贷」 vs 「数据银行（词库）」
- Word2Vec 一个词只能有一个向量 → 被迫取「平均义」
- 平均义的向量：既不像存钱，也不像河岸——两头不讨好

Word2Vec 的致命弱点



第四级：BERT 的做法（它怎么训练）

- **预训练：随机盖住 15% 的词，让 BERT 猜被盖住的是谁（掩码猜词）**
- 微调：用带标签的任务数据再训几轮，变成「懂你任务」的 BERT
- 结果：每个词的向量取决于它所在的整句话
- 同一个词在不同语境下是不一样的向量——一词多义被解决

① 预训练 = 读万卷书	② 微调 = 做你的题	③ 看整句，不看窗口
随机盖住 15% 的词（[MASK]） 让 BERT 猜被盖住的是谁	用带标签的任务数据 再训几轮，做你的任务	不像 Word2Vec 只看邻居 整句摊开，每个词扫全桌

预训练费钱费时（大厂做）；微调很便宜（几小时）→ 直接下载别人预训好的

BERT 的输入：三个向量加在一起

- **每个 token 用三个向量相加：词向量 + 位置向量 + 段向量**
- 词向量：这个词本身的意思（沿用 Word2Vec 的思想）
- 位置向量：这个词在第几个位置（顺序信息）
- 段向量：这句话属于前半还是后半（句对任务用）

BERT 为什么能看上下文：自注意力

- **自注意力：每个词同时「扫」句子里所有其他词，借线索定自己的含义**
- 双向：既看左边（前文）也看右边（后文）——Word2Vec 做不到
- 例：「风险」借「市场」与「上升」的线索，才能定是哪种风险
- 12 层堆叠逐层抽象：词 → 短语 → 句意——这套机制叫 Transformer

Word2Vec：只看窗口邻居
（前后 2 个词）

BERT：整句摊在桌上
每个词都能扫到全桌



线越粗 = 借的线索越多 → 所以「风险」在句里有了准确含义

什么时候用哪一级：选型决策表

任务	推荐级别	原因
快速文本分类、词表不大	词袋 / TF-IDF	快、可解释、够用
词语相似度、类比推理	Word2Vec	稠密向量、有语义方向
一词多义、句子级理解	BERT	上下文相关
大文档检索（RAG）	BERT 向量 + 向量库	句子级语义、规模大

HuggingFace 模型卡：三个必看字段

字段	看什么	为什么重要
支持语言	Chinese / multilingual	英文模型硬上中文=白做
下载量	downloads 数	量大=被验证过、坑少
许可证	Apache-2.0 / MIT / 商用限制	论文/商用要合规

本地加载代码逐行拆解

- **AutoTokenizer.from_pretrained(路径)：加载分词器**
- AutoModel.from_pretrained(路径)：加载模型权重
- 路径写本地目录 → 离线可用；写模型名 → 自动下载
- 输出是 last_hidden_state：每个 token 一行向量

实操演练清单：两条路各走一遍

- **路线 A（本地）：下载 FinBERT-zh → 本地加载 → 取句子向量**
- 路线 B（在线）：注册 API → 传句子 → 收向量
- 对照产出：同一句话、两条路的向量 → 算余弦相似度
- 收尾问题：哪个快？哪个贵？哪个数据不出内网？

现场看：同一个词，两个向量

句 A：苹果很好吃（食物）

v（苹果 • 食物语境）

```
0.2314  -0.1187
0.0952   0.0671
-0.1543  0.0876
-0.0342  0.1128
```

句 B：苹果发布新品（公司）

v（苹果 • 公司语境）

```
0.1986   0.1542
-0.0763  0.1105
0.2317  -0.0891
0.1742  -0.0526
```

余弦相似度 ≈ 0.68 ≠ 1

同一个词，两句话里是两个不同的向量——这就是 BERT 与 Word2Vec 的本质差别

四级台阶小结

级别	解决什么	还差什么
词袋	文字变数字	丢语序、词表爆炸
TF-IDF	压掉废话词	仍是无序计数
Word2Vec	稠密向量、语义关系	一词一义
BERT	上下文相关	算力要求高

03

向量模型实操

本地 vs 在线 • HuggingFace

本地还是在线：三条权衡轴线

	本地部署 台式机 + 本地模型	在线 API 云服务 + 接口调用
① 数据是否出内网	不出内网（安全）	出境（合规风险）
② 联网与费用	可离线 • 零边际成本	依赖网络 • 按量计费
③ 部署维护成本	要显存 • 自己管环境	开箱即用 • 免维护

课堂演示用本地；机器配置不齐时改在线——涉及未公开数据用本地，学习探索用在线

HuggingFace 是什么

- **模型界的 github，大部分开源向量模型都在这**
- 每个模型有模型卡：说明用途、语言、许可证
- 关键看三样：支持的语言、下载量、许可证是否允许商用
- 中文任务记得筛 Chinese 或 multilingual

演示：怎么找到想要的模型

huggingface.co/models：按条件筛到想要的中文模型

chinese finance bert　Search

筛选条件

Task: Feature Extraction

Language: Chinese

Library: transformers

Sort: Downloads

① FinBERT-zh (chinese finance bert)

downloads 12.4k · 中文金融预训练 · 许可证 Apache-2.0

② bert-base-chinese (通用中文 BERT)

downloads 88.1k · 通用中文 · 可商用

③ Chinese-MiniLM (轻量中文向量)

downloads 33.5k · 快、省显存 · 质量略降

关键看三样：支持的语言、下载量、许可证是否允许商用

中文任务记得筛 Chinese 或 multilingual——不要拿英文模型硬上中文

演示：下载与本地加载

两条路之一：下载到本地，离线加载

```
$ huggingface-cli download FinBERT-zh
Fetching 4 files ...
config.json ✓ (1.2 KB)
pytorch_model.bin ✓ (412 MB)
vocab.txt ✓ (108 KB)
Done. cached in ~/.cache/huggingface
```

模型目录结构：

```
├─ config.json 配置
├─ pytorch_model.bin 权重
├─ vocab.txt 词表
└─ tokenizer.json 分词器
```

```
from transformers import AutoTokenizer, AutoModel
model_name = './local_models/FinBERT-zh' # 本地路径，断网也能用
tok = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)
vec = model(**tok('公司经营稳健', return_tensors='pt')).last_hidden_state
```

本地加载 = 零网络依赖、零按量计费；代价是提前下载 + 自己管环境

两条路都走一遍的指令

- **本地：用刚才下载的中文模型，给下面这句话出向量**
- 在线：调用在线 API，给同一句话出向量
- 例子句子：「公司持续优化资产负债结构，经营性现金流保持稳定」
- 然后比较这两个向量的余弦相似度

看结果：两条路差多少

看结果：本地 BERT vs 在线 API，同一个句子

本地 BERT	在线 API
0.4123	0.4098
-0.2215	-0.2231
0.0872	0.0885
0.1531	0.1517
-0.0918	-0.0902
0.3345	0.3319
-0.1204	-0.1196
0.0687	0.0692

余弦相似度 = 0.998

两条路给出的向量高度一致 → 选哪条主要看成本与合规，不看效果

04

RAG 原理

四步流水线 • 检索与重排

幻觉机制：模型为什么敢一本正经地编

- **训练目标是「预测下一个词」，不是「查事实」**
- 没有知识约束 → 输出最「顺」的答案，顺 ≠ 对
- 模型不知道「自己不知道」——它没有校准机制
- 例：问「茅台 2024 净利润」→ 编一个像样的数字，语气还特别自信

幻觉 vs 检索错误：两种失败要分清

失败类型	表现	根源	对策
幻觉	答案没有来源，模型自己编	没证据约束	上 RAG，强制引用
检索错误	知识库有，但没找对	切片/召回问题	换切片策略、加重排

为什么需要 RAG：大模型读不到你的私有数据

- **模型只见过训练截止日之前的公开语料**
- 你的财报 PDF、内部制度、私有资料，它一概没见过
- 硬问会一本正经地编（幻觉）；先建知识库再问才是正道

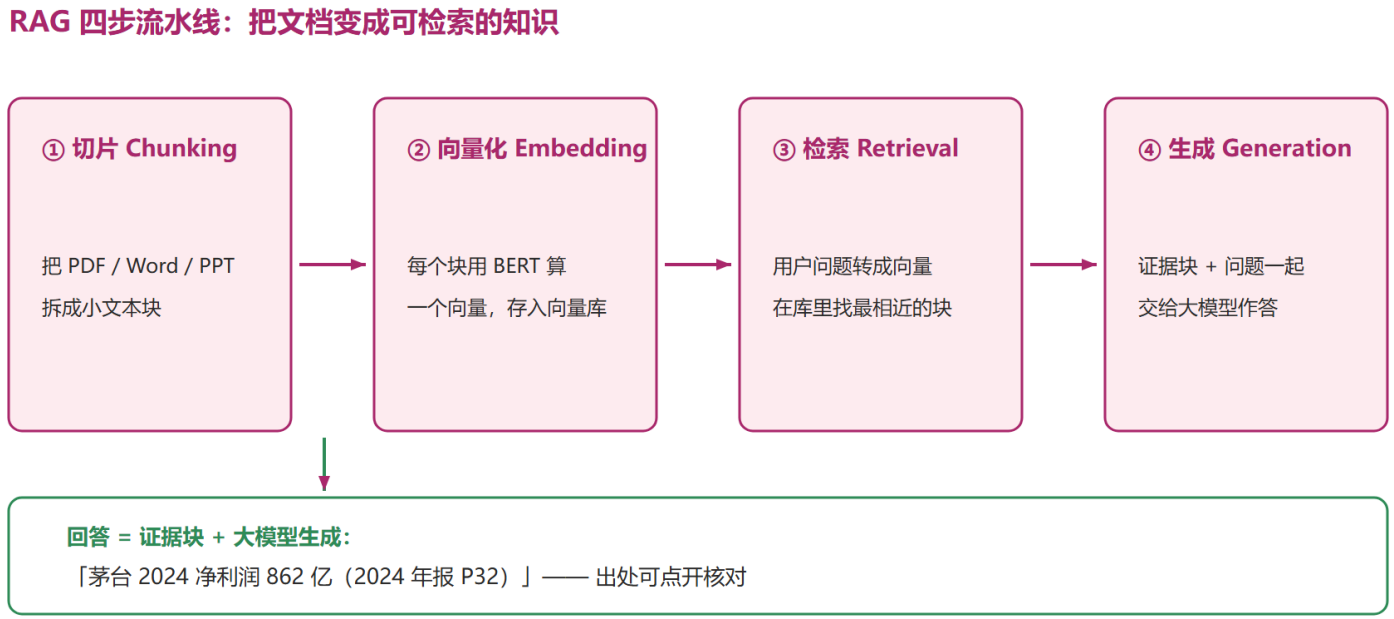


四步细节：每一步的输入和输出

步骤	输入	输出	关键问题
① 切片	整份 PDF / Word	文本块 chunks	块多大？切在哪？
② 向量化	每个 chunk	一个向量	用哪个 BERT 模型？
③ 检索	用户问题	Top-K 个 chunk	K 取多少？相似度怎么算？
④ 生成	问题 + 证据块	答案 + 引用出处	提示词怎么写？

RAG 是什么：四步流水线

- **RAG = Retrieval-Augmented Generation，检索增强生成**
- 先检索「相关证据片段」，再让大模型基于证据作答
- 四个动作：切碎文档 → 向量化 → 检索 → 拼接生成
- 回答自带出处、可逐条核查——正好呼应下午「追溯结论」红线



切片策略：块切得好不好，决定检索准不准

策略	做法	优点	缺点
固定长度	每 300–500 token 一刀	简单、可控	可能切断句子/表格
按结构切	按标题/段落/章节	保留语义边界	长段落可能超长
表格/图片	转 Markdown / OCR / VLM 描述	不丢信息	需要额外解析

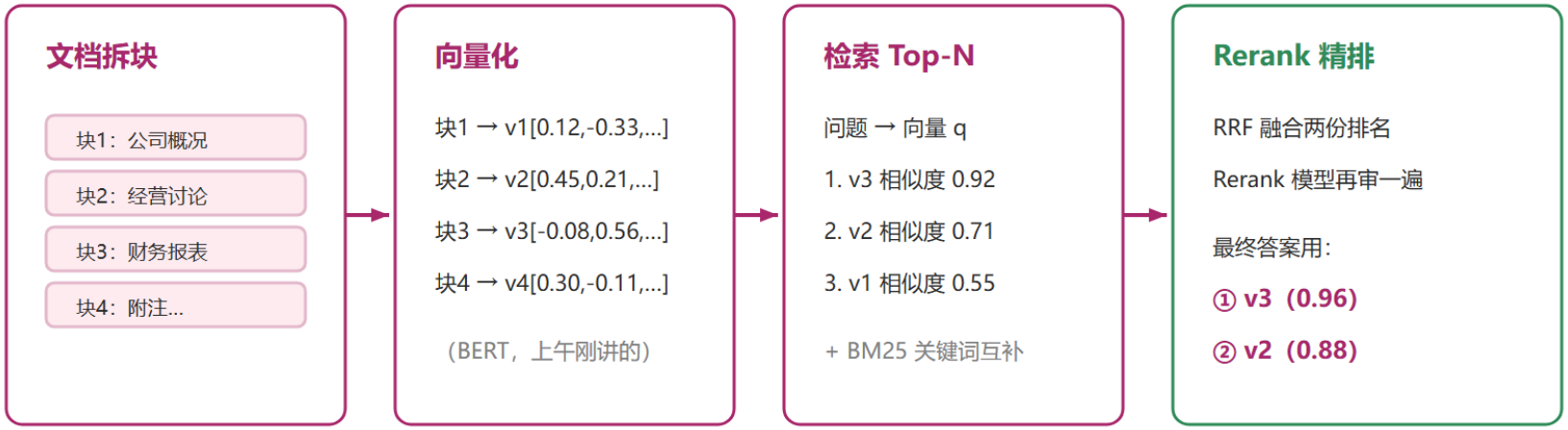
向量库怎么选：四个常用选项

方案	类型	适合场景	上手难度
FAISS	库（Meta）	单机百万级、研究	低
Qdrant	向量数据库	中小团队、好用	低
Milvus	分布式向量库	企业级、海量	中
pgvector	Postgres 插件	已有 Postgres、顺路	低

切片 • 向量化 • 检索 • 重排

- **切片：按标题 / 段落 / 固定长度拆；表格、图片单独处理（转 MD / OCR / VLM）**
- 向量化：用上午 BERT 那一套，每个块算一个向量，进向量库
- 检索：向量相似度找语义近邻 + BM25 关键词精确命中互补
- 重排：RRF 融合两份排名，Rerank 模型再精排一遍

关键动作拆解：切块 → 向量 → 检索 → 精排



余弦相似度手算：到底怎么算「像不像」

- **余弦 = 两向量点积 ÷ 模长乘积，范围 [-1，1]**
- 例：v=[1,0]（「净利润」）与 w=[0.9,0.3]（「利润」）→ 相似 0.95
- 例：v=[1,0] 与 z=[0,1]（「负债」）→ 相似 0，正交无关
- 为什么用余弦：只看方向不看长度——词频高低不该影响相似度

提升召回：查询改写 + 多路检索

- **查询改写：把「茅台今年赚多少」拆成「净利润」「2024」「贵州茅台」**
- 多路召回：向量相似 + BM25 关键词 + 知识图谱各出一份候选
- RRF 融合：三份排名按名次倒序合并，取交集靠前的
- 效果：单路漏掉的，多路往往能补回来

怎么评估一个 RAG：两个层面

- **检索层面：Top-K 里有没有正确答案（命中率@K）**
- 生成层面：答案是否忠于证据（忠实度），与问题相关吗（相关度）
- 快速评估法：抽 50 条真问题人工打分，看两个比例
- 呼应下午红线：每条结论都能溯源——评估就是在查「溯源率」

主流 RAG 方案：三条路怎么选

路线	代表	适合谁	特点
代码自建	LangChain / LlamaIndex	要深度定制的开发者	最灵活，全自己写
开源平台	RAGFlow / Dify / MaxKB / QAnything	想快速上线的团队	有界面，拖拽配置
开箱即用应用	AnythingLLM	个人/课堂、不想折腾环境	桌面版零配置，内置一切

05

AnythingLLM 与落地

轻量底座 • 安装演示 • 数据基座收束

AnythingLLM 安装三步：下载安装即用

步骤	做什么	产出
① 下载桌面版	anythingllm.com → 选 Windows/Mac/Linux 安装包	本地应用（约 500MB）
② 连本地模型	选 Ollama（自动探测 127.0.0.1:11434）→ 聊天模型 + 嵌入模型	可对话的 LLM
③ 建工作区传文档	新建 Workspace → 拖入 PDF/Word	可问答的 RAG 实例

AnythingLLM 演示全流程：问财报

- **建工作区：新建「2024 年报库」➜ 拖入 5 份 PDF ➜ 自动切片 + 嵌入（几十秒）**
- 问答：「2024 年茅台净利润是多少？」
- 看结果：答案带引用，点开即见原文段落（source citation）
- 追问：「和 2023 年比增长多少？」——多轮追问同样带引用

AnythingLLM：本地优先的 RAG 桌面应用

- **Mintplex Labs 开源，GitHub 60K+ Star，MIT 协议可商用**
- 定位：all-in-one 本地优先 AI 应用——文档问答 + AI Agent 开箱即用
- 三个亮点：内置向量库 LanceDB（零配置）/ 内置嵌入模型 / 桌面版三平台
- 桌面版约 500MB、内存 2-4GB——轻量够用，普通笔记本带得动



AnythingLLM 和 WorkBuddy 怎么打通：MCP

- **AnythingLLM 支持 MCP 兼容（MCP-compatibility）：知识库可暴露成工具**
- WorkBuddy 连上后，Agent 可以直接「查知识库」再回答
- 例：让 WorkBuddy「查一下工作区里的年报，算三率变化」——它自己去查
- 呼应 Day1 上午 MCP 概念：工具 = 给 Agent 接上知识层

AnythingLLM 全能力速览

环节	AnythingLLM 的做法
文档解析	PDF / TXT / DOCX / MD 等，拖拽即解析（内置 pipeline），支持粘贴网址
向量库	默认内置 LanceDB（零配置）；可换 Qdrant / Milvus / Chroma / Pinecone
嵌入模型	内置 Native Embedder（默认）；可换 Ollama nomic-embed-text 等
生成模型	30+ 家：Ollama / LM Studio 本地，OpenAI / DeepSeek / Gemini / Moonshot 云
特色	Workspace 工作区隔离 • AI Agent（网页/SQL/图表）• MCP 兼容 • 定时任务
部署	桌面版（单用户零配置）/ Docker（多用户）/ 云模板（Railway/Render）

AnythingLLM 的两个进阶点：工作区隔离 + Agent

- **工作区隔离：每个项目一个 Workspace，A 项目的文档 B 看不到（防串库）**
- AI Agent：网页浏览、SQL 查库、生成图表，拖拽搭流程（无代码）
- 定时任务：按 cron 周期自动跑提示词/抓取（呼应 Day1 下午定时轮询）
- 边界提醒：图谱/Rerank 这类企业级功能，更重的企业级平台才有

最小实现：FAISS 一行代码跑通检索

- index = faiss.IndexFlatIP(dim)：建索引
- index.add(vecs)：向量入库
- D, I = index.search(query_vec, k=5)：一次检索 Top-5
- 三行代码就能演示 RAG 的检索环节（数据小、理解深）

自建 vs 应用：成本与控制的权衡

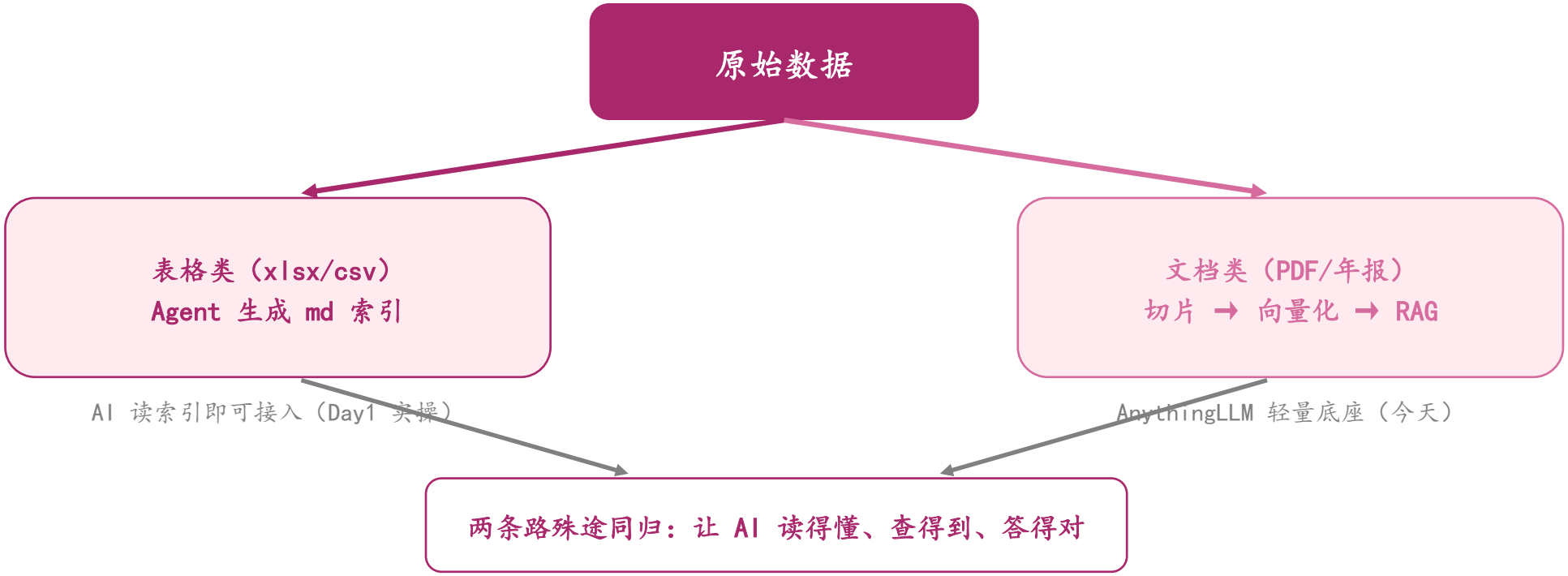
维度	自建（LangChain+FAISS）	桌面应用（AnythingLLM）
上手成本	高：切片/向量库/编排全自己写	极低：下载安装即用
可控性	全可控、可定制	跟随社区迭代
功能	单一 RAG	RAG + Agent + 工作区隔离
适合	学习原理、特殊需求	快速搭个人/课堂知识库

落地清单：从 0 到 1 搭自己的数据基座

- **① 盘点数据：哪些是表格（→md 索引），哪些是文档（→RAG）**
- ② 表格类：让 Agent 检查数据、生成索引 md（Day1 上午那套）
- ③ 文档类：装 AnythingLLM（轻量）或自建（切片+向量库）
- ④ 验证：拿 10 个真问题测，看命中与溯源
- ⑤ 更新机制：新文档进来怎么进库（定时同步/手动上传）

数据基座全景：两条路收束

- **表格类数据：Agent 生成 md 索引，AI 直接接入（Day1 上午实操）**
- 文档类数据：切片 → 向量化 → RAG；AnythingLLM 就是现成的轻量桌面底座
- 两条路殊途同归：让 AI 读得懂、查得到、答得对
- 下午预告：用 LLM 直接抽取 + 合成动态报告，把知识真正用起来



06

五维文化实战

复现论文：从种子词到文化画像

现在做一件真研究做过的事

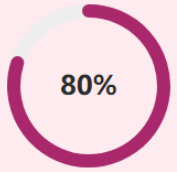
- **参考文献：Li, Mai, Shen \& Yan (2020), RFS**
- 题目：用机器学习测度企业文化
- 我们今天要复现它的核心方法
- 但语料不一样——这点后面专门讲

五个文化维度从哪来

五个文化维度：从官网词频里长出来

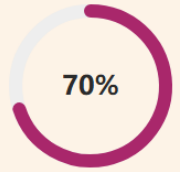
维度不是我们拍的——来自 S&P 500 公司官网的高频词聚类

创新
Innovation



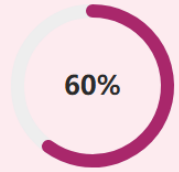
研发、新产品、专利

诚信
Integrity



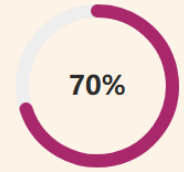
诚实、合规、透明

品质
Quality



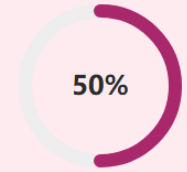
质量、标准、精工

尊重
Respect



员工、客户、包容

协作
Teamwork

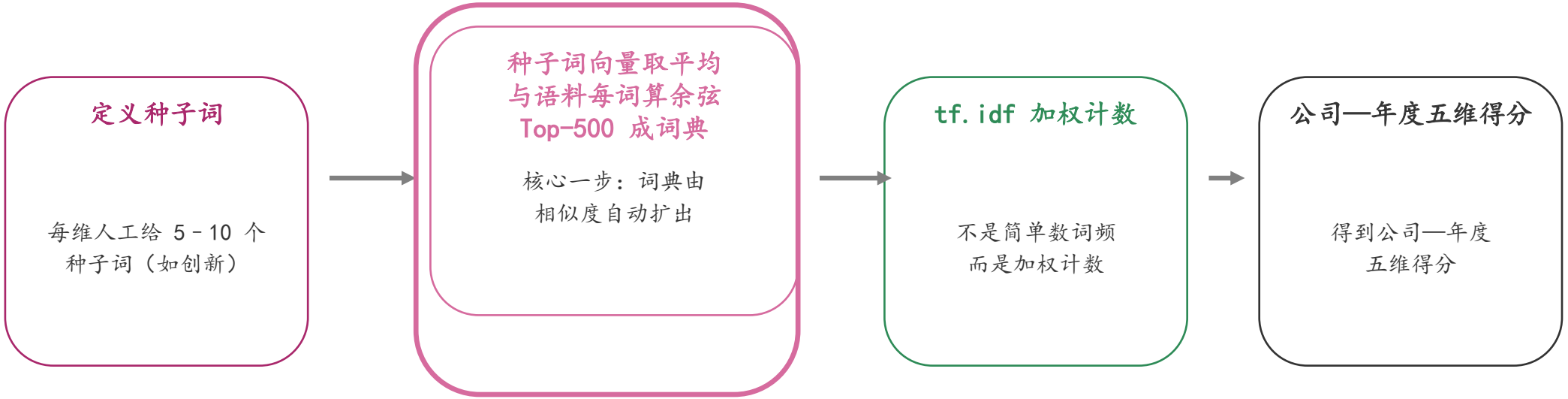


合作、团队、共赢

提及率 = 该词在官网文本里出现的相对频率；越高代表公司越常把这条价值挂嘴边

论文用这套维度给上千家公司打分——今天我们用中文年报复现一次

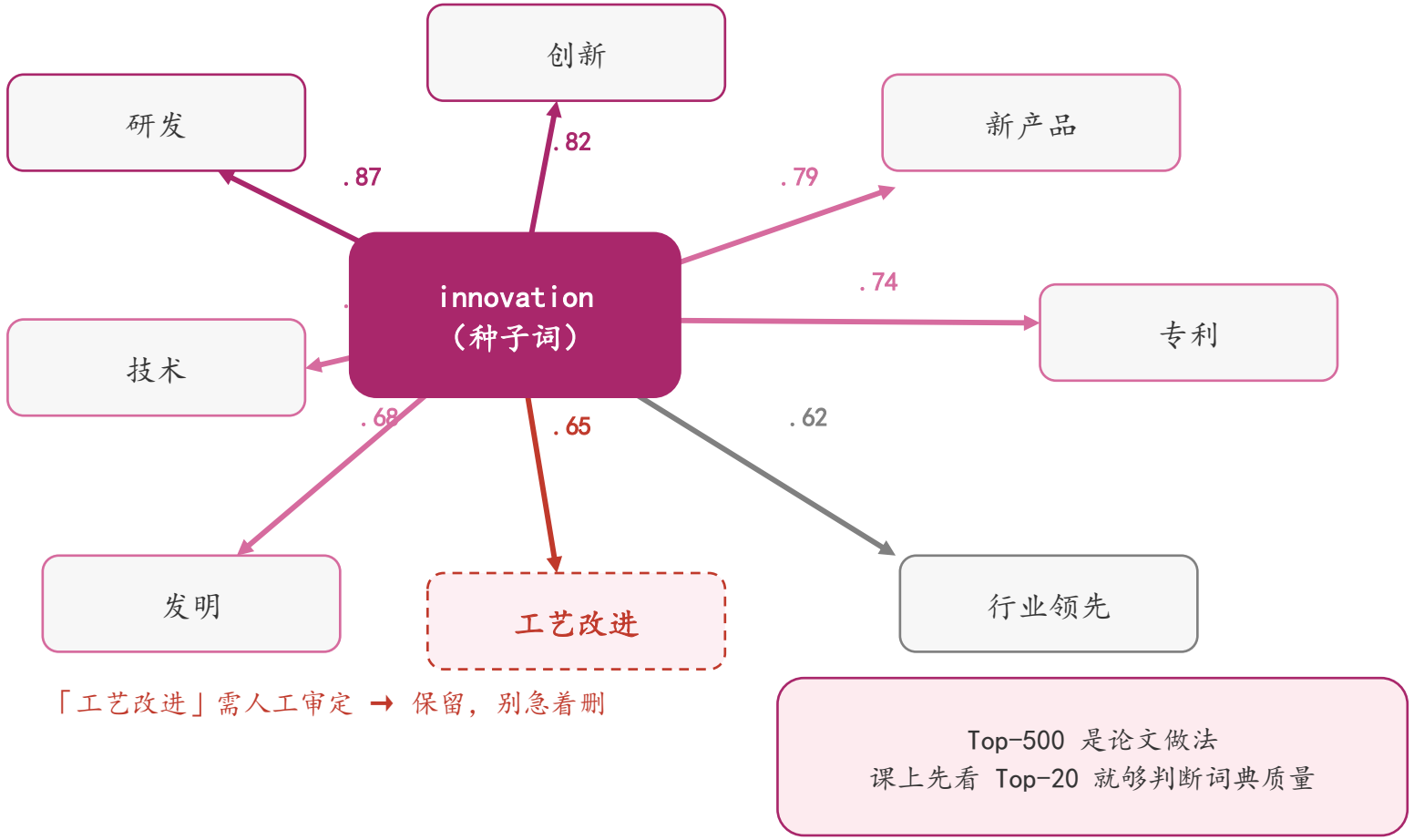
方法四步：怎么从种子词得到得分



方法来自 Li, Mai, Shen & Yan (2020), RFS

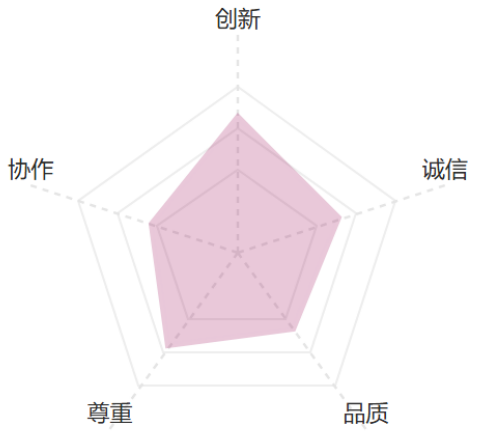
演示：从种子词扩出一整部词典

半监督：人给一点点指引
模型自己学出一大张词表

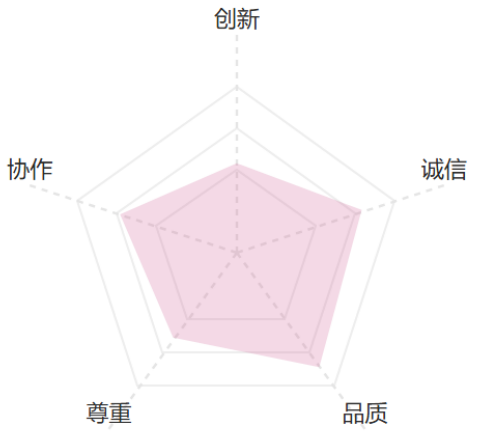


得到分数：每家公司的文化画像

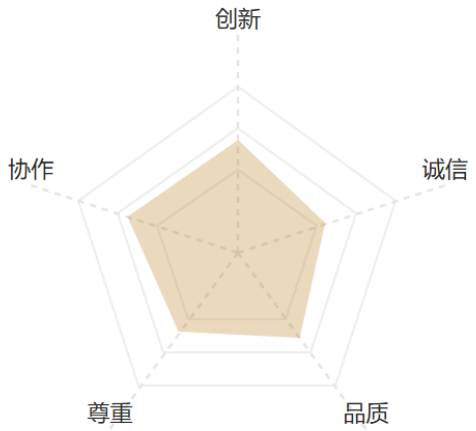
得到分数：三家公司的文化画像（示意）



公司 A



公司 B



公司 C

年报 MD&A；与论文原版（电话会 QA 段）不同，仅描述性参考 年报 MD&A；与论文原版（电话会 QA 段）不同，仅描述性参考 年报 MD&A；与论文原版（电话会 QA 段）不同，仅描述性参考

形状差异 = 文化差异；同色系不同明度便于对比，图例统一

★ 关键交代：我们的语料和论文不一样

关键交代：我们的语料和论文不一样

论文原版

① 209,480 份业绩电话会 ✓

② 只用即兴问答 QA 段，
不用管理层宣读段

③ 经理人没法挑话题（Lee 2016），
更接近真实

我们的做法

① 年报 MD&A 文本 ⚠

② 管理层精心撰写的
事后自述

③ 粉饰空间大

方法相同，语料不同 → 结论只能当「描述性证据」，不能当论文原结论

这一步交代不是谦虚——是学术诚信：实验边界必须说清楚

★ 论文说过：MD&A 这条路不如 QA 段

论文说过的：MD&A 这条路不如 QA 段

论文第 5 页把「同一套词嵌入方法用于年报 MD&A」列为对照方案 (c)

——他们早试过我们要走的路，结论已经写在论文里

QA 段（即兴）

即兴 · 不可预演

→

MD&A（撰写）

事后撰写 · 可反复润色（条更长 = 粉饰空间更大）

论文验证结果：QA 方案显著优于 MD&A 方案

→ 我们的得分只能作描述性 evidence，不能当作论文原结论

教学价值：用真实限制把「方法迁移的边界」讲明白——比拿到满分更值钱

这个妥协意味着什么

- **MD&A 是管理层精心撰写的事后自述，粉饰空间比即兴问答大得多**
- QA 段经理人没法挑话题，更接近真实想法
- 所以我们的得分只能作描述性 evidence
- 不能像原论文那样直接拿去做因果推断

教学价值恰恰在这里

- **同一个模型、同一套词典，换了语料，结论强度就变了**
- 方法能不能迁移，不只看代码能不能跑通
- 要看原始方法的成立条件，在你这里还成不成立
- 这就是本课程反复强调的「判断在前」

07

小结

上午回顾 + 下午预告